

LAPORAN PRAKTIKUM PRAKTIKUM STRUKTUR DATA

Disusun oleh

Nama : Muhammad Yusuf Insy

NIM : 2511532003

Dosen Pengampu : DR. Wahyudi,S.T,M.T.

Asisten Praktikum : jovan



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN 2025

Daftar Isi

BAB 1 PENDAHULUAN	3
1. Latar Belakang	3
2. Tujuan	3
3. Manfaat	3
BAB 2 PEMBAHASAN.....	4
1.1 isi.....	4
BAB 3 PENUTUP	11
1.1. Kesimpulan	11
Daftar Isi	12

BAB 1 PENDAHULUAN

1. Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data secara efisien. Salah satu struktur data dasar yang sering digunakan adalah **Single Linked List (SLL)**.

Single Linked List adalah struktur data linear yang terdiri dari node-node yang saling terhubung menggunakan pointer, di mana setiap node memiliki dua bagian yaitu data dan pointer ke node berikutnya. Struktur ini sangat berguna untuk operasi dinamis seperti penambahan, penghapusan, dan pencarian data.

Dalam praktikum ini, dilakukan implementasi Single Linked List menggunakan bahasa pemrograman Java untuk memahami konsep dasar operasi insert, delete, search, dan traversal secara langsung melalui kode program.

2. Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Single Linked List.
2. Mengimplementasikan operasi insert pada SLL (depan, belakang, dan posisi tertentu).
3. Mengimplementasikan operasi delete pada SLL (head, tail, dan posisi tertentu).
4. Mengimplementasikan operasi pencarian (search) dan penelusuran (traversal).
5. Melatih pemahaman pointer dan struktur node dalam Java.

3. Manfaat

Manfaat dari praktikum ini antara lain:

1. Menambah pemahaman tentang struktur data linked list.
2. Meningkatkan kemampuan logika pemrograman.
3. Mampu mengelola data secara dinamis tanpa array statis.
4. Menjadi dasar untuk mempelajari struktur data lanjutan seperti stack, queue, dan tree.

BAB 2 PEMBAHASAN

1.1 isi

1. Kelas NodeSLL_2511532003 (Struktur Node)

Kode:

```
int data_2003;  
NodeSLL_2511532003 next;  
  
public NodeSLL_2511532003(int data)  
{  
    this.data_2003 = data;  
    this.next = null;  
}
```

Penjelasan:

Kelas ini adalah **struktur dasar dari Single Linked List (SLL)**.

a. Variabel data_2003

- Menyimpan nilai/data yang akan disimpan di dalam node.
- Contoh: 1, 2, 3, 10, dll.

b. Variabel next

- Merupakan pointer (referensi) ke node berikutnya.
- Jika next = null, berarti node tersebut adalah **node terakhir**.

c. Constructor

```
this.data_2003 = data;  
this.next = null;
```

- Saat node dibuat, langsung diisi data.
- Pointer next otomatis kosong (belum terhubung ke node lain).

Fungsi utama:

- Menjadi “blok bangunan” Linked List.

- Setiap node akan saling terhubung membentuk rantai.

2. Kelas tambahSLL_2511532003 (Insert / Penambahan Data)

♦ a. Insert di depan

✦ Kode:

```
new_node_2003.next = head_2003;  
return new_node_2003;
```

🔍 Penjelasan:

- Node baru dibuat.
- Pointer next node baru menunjuk ke head lama.
- Node baru menjadi head baru.

✦ Proses:

1. Buat node baru
2. Sambungkan ke head lama
3. Ganti head

🧠 Kompleksitas:

- $O(1) \rightarrow$ sangat cepat

♦ b. Insert di belakang

✦ Kode:

```
while (last_2003.next != null) {  
    last_2003 = last_2003.next;  
}  
last_2003.next = newNode_2003;
```

🔍 Penjelasan:

- Mulai dari head.

- Traversal sampai node terakhir (next = null).
- Node terakhir dihubungkan ke node baru.

Proses:

1. Cari node terakhir
2. Tambahkan node baru di akhir

Kompleksitas:

- $O(n) \rightarrow$ harus jalan satu per satu

c. Insert di posisi tertentu

Kode inti:

```
while (position_2003-- != 0) {
    if (position_2003 == 1) {
        newNode_2003.next = headNode_2003.next;
        headNode_2003.next = newNode_2003;
    }
    headNode_2003 = headNode_2003.next;
}
```

Penjelasan:

- Program mencari posisi tertentu.
- Pointer digeser sampai posisi target.
- Node baru disisipkan di tengah list.

Proses:

1. Traversal ke posisi
2. Simpan node setelahnya
3. Sisipkan node baru
4. Hubungkan kembali pointer

Kompleksitas:

- $O(n)$

◆ d. Print List

✚ Kode:

```
while (curr_2003 != null) {  
    System.out.print(curr_2003.data_2003 + "-->");  
    curr_2003 = curr_2003.next;  
}
```

🔍 Penjelasan:

- Menampilkan seluruh isi Linked List.
- Traversal dari head sampai null.

🧠 Fungsi:

- Debugging
- Melihat isi data

3. Kelas HapusSLL_2511532003 (Delete / Penghapusan Data)

◆ a. Hapus Head

✚ Kode:

```
head_2003 = head_2003.next;
```

🔍 Penjelasan:

- Head digeser ke node berikutnya.
- Node pertama otomatis tidak digunakan lagi.

✚ Ilustrasi:

1 → 2 → 3

jadi:

2 → 3

🧠 Kompleksitas:

- $O(1)$

♦ b. Hapus node terakhir

✚ Kode:

```
while (secondLast_2003.next.next != null) {
    secondLast_2003 = secondLast_2003.next;
}
secondLast_2003.next = null;
```

🔍 Penjelasan:

- Cari node sebelum terakhir.
- Putuskan hubungan ke node terakhir.

✚ Ilustrasi:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

jadi:

$1 \rightarrow 2 \rightarrow 3$

🧠 Kompleksitas:

- $O(n)$

♦ c. Hapus berdasarkan posisi

✚ Kode:

```
prev_2003.next = temp_2003.next;
```

🔍 Penjelasan:

- Node target dilewati.
- Pointer langsung disambungkan ke node setelahnya.

✚ Ilustrasi:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

hapus posisi 2:

1 → 3 → 4

Kompleksitas:

- $O(n)$

◆ d. Print List

Kode:

```
while (curr_2003 != null) {  
    System.out.print(curr_2003.data_2003);  
    curr_2003 = curr_2003.next;  
}
```

Penjelasan:

- Menampilkan isi setelah proses delete.

4. Kelas PencarianSLL_2511532003 (Search & Traversal)

◆ a. Search (Linear Search)

Kode:

```
if (curr_2003.data_2003 == key_2003)  
    return true;
```

Penjelasan:

- Setiap node dibandingkan dengan nilai key.
- Jika cocok → return true.

Proses:

1. Mulai dari head
2. Bandingkan data
3. Jika sama → ketemu
4. Jika tidak → lanjut

Kompleksitas:

- $O(n)$

◆ b. Traversal

Kode:

```
while (curr_2003 != null) {  
    System.out.print(curr_2003.data_2003);  
    curr_2003 = curr_2003.next;  
}
```

Penjelasan:

- Menampilkan semua isi Linked List.
- Tidak mengubah data.

Fungsi:

- Menampilkan isi list sebelum pencarian

BAB 3 PENUTUP

1.1. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data yang terdiri dari node-node yang saling terhubung menggunakan pointer. Setiap node memiliki data dan referensi ke node berikutnya.

Operasi pada Single Linked List seperti insert, delete, search, dan traversal memiliki cara kerja yang berbeda-beda tergantung pada posisi node yang diakses. Operasi insert dan delete di bagian head memiliki kompleksitas $O(1)$, sedangkan operasi pada posisi lain membutuhkan traversal sehingga memiliki kompleksitas $O(n)$.

Dengan praktikum ini, pemahaman mengenai konsep pointer, node, dan manipulasi data dinamis dalam Java menjadi lebih jelas dan mudah dipahami.

Daftar Isi

Abdul Kadir. *Dasar Pemrograman Java*. Yogyakarta: Andi Offset.

Robert Lafore. *Data Structures and Algorithms in Java*. Sams Publishing.

Oracle Documentation. “Java Language Specification”. <https://docs.oracle.com/javase/>

Modul Praktikum Struktur Data – Universitas / Kampus (Pekan 5: Single Linked List).

GeeksforGeeks. “Linked List in Data Structure”. <https://www.geeksforgeeks.org/linked-list-data-structure/>